

PySpark spark.conf

Azure Data Engineer · Complete Command Reference

Spark 3.x · Azure Databricks · ADLS Gen2 · Delta Lake · Synapse

This reference covers every **spark.conf.set()** / **spark.conf.get()** command you need as an Azure Data Engineer — from cluster sizing to ADLS authentication, Delta Lake, AQE tuning, serialization, and monitoring.

Sections Covered

- 1. SparkSession Initialization
- 2. Dynamic Resource Allocation
- 3. Azure Data Lake Storage (ADLS Gen2)
- 4. Shuffle & Performance Tuning
- 5. Delta Lake & Azure Databricks
- 6. Azure Synapse / SQL / Event Hubs
- 7. Serialization & Compression
- 8. Logging, Monitoring & Lineage
- 9. Reading spark.conf Values

■ 1. SparkSession Initialization

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.app.name", "MyAzureApp")</code>	<code>spark.app.name = MyAzureApp</code>	Set the application name shown in Spark UI / Azure Monitor logs.
<code>spark.conf.set("spark.master", "yarn")</code>	<code>spark.master = yarn local[*] spark://host:port</code>	Execution mode. Use 'yarn' on Databricks/HDInsight; 'local[*]' for dev.
<code>spark.conf.set("spark.executor.memory", "8g")</code>	<code>spark.executor.memory = 4g 8g 16g</code>	RAM per executor. Tune based on Azure VM SKU (e.g. Standard_DS3_v2 = 14 GB).
<code>spark.conf.set("spark.driver.memory", "4g")</code>	<code>spark.driver.memory = 2g 4g 8g</code>	RAM for the driver process. Increase when collecting large DataFrames.
<code>spark.conf.set("spark.executor.cores", "4")</code>	<code>spark.executor.cores = 2 4 8</code>	vCores per executor. Balance with memory to avoid GC pressure.
<code>spark.conf.set("spark.executor.instances", "10")</code>	<code>spark.executor.instances =</code>	Static number of executors. Prefer dynamic allocation on cloud clusters.

■ 2. Dynamic Resource Allocation

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.dynamicAllocation.enabled", "true")</code>	<code>true false</code>	Auto-scale executors based on workload. Recommended for Azure Databricks.
<code>spark.conf.set("spark.dynamicAllocation.minExecutors", "2")</code>	<code>minExecutors =</code>	Floor for auto-scaling. Keeps warm executors to reduce cold-start latency.
<code>spark.conf.set("spark.dynamicAllocation.maxExecutors", "50")</code>	<code>maxExecutors =</code>	Cap on auto-scaling to control Azure compute cost.
<code>spark.conf.set("spark.dynamicAllocation.executorIdleTimeout", "60s")</code>	<code>executorIdleTimeout = 60s 120s</code>	Release idle executors after this period to free cluster resources.
<code>spark.conf.set("spark.dynamicAllocation.schedulerBacklogTimeout", "1s")</code>	<code>schedulerBacklogTimeout = 1s 5s</code>	Request new executors when tasks are pending longer than this.
<code>spark.conf.set("spark.shuffle.service.enabled", "true")</code>	<code>true false</code>	Required for dynamic allocation. Maintains shuffle files when executors are removed.

■ 3. Azure Data Lake Storage (ADLS Gen2)

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("fs.azure.account.auth.type..dfs.core.windows.net", "OAuth")</code>	OAuth SharedKey SAS	Authentication method for ADLS Gen2. OAuth (Service Principal) is best practice.
<code>spark.conf.set("fs.azure.account.oauth.provider.type...", "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")</code>	ClientCredsTokenProvider	Token provider class for Service Principal (client credentials) auth flow.
<code>spark.conf.set("fs.azure.account.oauth2.client.id...", "")</code>	<code>spark.conf.set(..., dbutils.secrets.get(scope, key))</code>	Application (Client) ID of the Azure AD Service Principal. Use Key Vault secrets.
<code>spark.conf.set("fs.azure.account.oauth2.client.secret...", dbutils.secrets.get("scope", "sp-secret"))</code>	Never hardcode secrets – always use <code>dbutils.secrets</code> or Azure Key Vault.	Client secret for the Service Principal. Rotated via Azure AD app registrations.
<code>spark.conf.set("fs.azure.account.oauth2.client.endpoint...", "https://login.microsoftonline.com/oauth2/token")</code>	tenant-id from Azure Active Directory > Overview	OAuth2 token endpoint. Replace with your AAD Tenant ID.
<code>spark.conf.set("fs.azure.account.key..dfs.core.windows.net", "")</code>	Shared Key auth – use only for dev/test.	Account key authentication. Simpler but less secure than OAuth2 with SP.

■ 4. Shuffle & Performance Tuning

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.sql.shuffle.partitions", "200")</code>	Default 200. Use 2-3× executor cores for large joins/aggregations.	Controls parallelism of wide transformations (joins, groupBy). Critical for perf.
<code>spark.conf.set("spark.default.parallelism", "400")</code>	Typically 2× total executor cores	Default partition count for RDD operations. Set to 2× total vCores.
<code>spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50mb")</code>	-1 to disable 10mb 50mb 100mb	Broadcast smaller table in joins. Set -1 to disable for skewed data.
<code>spark.conf.set("spark.sql.adaptive.enabled", "true")</code>	true (default in Spark 3.x)	Adaptive Query Execution: dynamically optimizes shuffle partitions & joins.
<code>spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")</code>	true false	AQE merges small shuffle partitions post-execution to reduce overhead.
<code>spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")</code>	true false	AQE detects and splits skewed partitions automatically. Very useful on ADLS data.
<code>spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128mb")</code>	64mb 128mb 256mb	Target size for coalesced partitions after AQE optimization.
<code>spark.conf.set("spark.memory.fraction", "0.6")</code>	0.6 (default)	Fraction of JVM heap for execution + storage. Remaining 0.4 for user data.
<code>spark.conf.set("spark.memory.storageFraction", "0.5")</code>	0.5 (default)	Within memory.fraction, portion reserved for caching (RDD/DataFrame cache).

■ 5. Delta Lake & Azure Databricks

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.databricks.delta.preview.enabled", "true")</code>	true false	Enable Delta Lake preview features in Azure Databricks workspace.
<code>spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")</code>	true false	Auto-optimize write file sizes for Delta tables (Databricks-specific).
<code>spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")</code>	true false	Trigger background compaction after writes to reduce small file problem.
<code>spark.conf.set("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")</code>	Required for open-source Delta Lake outside Databricks.	Registers Delta SQL extensions (MERGE, CONVERT TO DELTA, etc.).
<code>spark.conf.set("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")</code>	Required alongside sql.extensions for OSS Delta.	Sets Delta as the default catalog for v2 table operations.
<code>spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")</code>	true false	Allow schema evolution on Delta MERGE operations — useful for CDC pipelines.

■ 6. Azure Synapse / SQL / Event Hubs

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.synapse.linkedService.name", "")</code>	Linked Service name from Synapse workspace	Configure Synapse linked service for reading/writing Synapse Analytics pools.
<code>spark.conf.set("spark.cosmos.accountEndpoint", "https://.documents.azure.com:443/")</code>	Cosmos DB endpoint URI from Azure portal.	Configure Azure Cosmos DB connector endpoint for Spark reads/writes.
<code>spark.conf.set("spark.eventhubs.connectionString", conn_str)</code>	Use dbutils.secrets for connection string.	Azure Event Hubs connection string for structured streaming ingestion.
<code>spark.conf.set("spark.eventhubs.maxEventsPerTrigger", "10000")</code>	Tune based on throughput units provisioned.	Rate-limit micro-batch size when reading from Event Hubs streams.

■ 7. Serialization & Compression

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")</code>	KryoSerializer (faster) JavaSerializer (default)	Kryo is significantly faster and smaller. Recommended for all production jobs.
<code>spark.conf.set("spark.kryo.registrationRequired", "false")</code>	false (recommended) true (strict)	When false, unregistered classes still serialize. Set true for debugging.
<code>spark.conf.set("spark.sql.parquet.compression.codec", "snappy")</code>	snappy gzip zstd lz4 none	Compression for Parquet output. Snappy is default; zstd gives better ratio.
<code>spark.conf.set("spark.sql.orc.compression.codec", "zlib")</code>	zlib snappy lz4	Compression codec for ORC files. zlib gives best compression ratio.
<code>spark.conf.set("spark.io.compression.codec", "lz4")</code>	lz4 snappy zstd	Codec for internal shuffle/broadcast data compression. lz4 is fastest.

■ 8. Logging, Monitoring & Lineage

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>spark.conf.set("spark.eventLog.enabled", "true")</code>	true false	Enable Spark event logging for Spark History Server / Azure Monitor.
<code>spark.conf.set("spark.eventLog.dir", "abfss://logs@.dfs.core.windows.net/spark-logs")</code>	ADLS Gen2 or Blob Storage path	Destination for Spark event logs. Point to ADLS for long-term retention.
<code>spark.conf.set("spark.sql.queryExecutionListeners", "")</code>	Comma-separated listener class names	Attach custom QueryExecutionListeners for metrics / Azure App Insights.
<code>spark.conf.set("spark.databricks.passthrough.enabled", "true")</code>	Databricks-specific. Requires AAD Credential Passthrough enabled.	Pass end-user Azure AD identity through to ADLS for row-level security.

■ 9. Reading spark.conf Values

spark.conf.set(KEY , ...)	Common Value(s)	Description / Azure Tip
<code>val = spark.conf.get("spark.executor.memory")</code>	Returns current value as string.	Read any config at runtime — useful for logging/debugging config state.
<code>all_conf = spark.sparkContext.getConf().getAll()</code>	Returns list of (key, value) tuples.	Dump all active Spark configurations. Filter for 'spark.' or 'fs.azure.' keys.
<code>spark.conf.unset("spark.sql.shuffle.partitions")</code>	Resets to default value.	Remove a previously set config, reverting to Spark/cluster default.
<code>spark.conf.isModifiable("spark.executor.memory")</code>	Returns True/False	Check if a config can still be changed at runtime (some require restart).

■ 10. Boilerplate — Azure Databricks Session Init

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
    .appName("AzureDataEngineerJob")
    .config("spark.sql.adaptive.enabled", "true")
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true")
    .config("spark.sql.adaptive.skewJoin.enabled", "true")
    .config("spark.sql.shuffle.partitions", "400")
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    .config("spark.sql.parquet.compression.codec", "snappy")
    .config("spark.dynamicAllocation.enabled", "true")
    .config("spark.dynamicAllocation.minExecutors", "2")
    .config("spark.dynamicAllocation.maxExecutors", "50")
    .config("spark.shuffle.service.enabled", "true")
    .getOrCreate())

# ADLS Gen2 – Service Principal (OAuth)
ACCOUNT = "myadlsaccount"
TENANT = "<tenant-id>"
CLIENT = dbutils.secrets.get("kv-scope", "sp-client-id")
SECRET = dbutils.secrets.get("kv-scope", "sp-client-secret")

spark.conf.set(f"fs.azure.account.auth.type.{ACCOUNT}.dfs.core.windows.net", "OAuth")
spark.conf.set(f"fs.azure.account.oauth.provider.type.{ACCOUNT}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set(f"fs.azure.account.oauth2.client.id.{ACCOUNT}.dfs.core.windows.net", CLIENT)
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{ACCOUNT}.dfs.core.windows.net", SECRET)
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{ACCOUNT}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{TENANT}/oauth2/token")

# Delta Lake (OSS – outside Databricks)
spark.conf.set("spark.sql.extensions",
    "io.delta.sql.DeltaSparkSessionExtension")
spark.conf.set("spark.sql.catalog.spark_catalog",
    "org.apache.spark.sql.delta.catalog.DeltaCatalog")
```

- Always store secrets in Azure Key Vault and access via `dbutils.secrets.get()`.
- Use `spark.conf.get(key)` at runtime to verify configs are applied correctly.
- AQE (`spark.sql.adaptive.enabled`) is ON by default in Spark 3.2+; leave it enabled.
- Set `shuffle.partitions` to 2-3× total executor vCores for large aggregations.
- On Databricks, prefer cluster-level configs in Cluster UI over per-notebook `spark.conf.set()`.
- `spark.conf.isModifiable()` returns `False` for configs that require a session restart.